Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ»
(ТУСУР)
Кафедра автоматизированных систем управления (АСУ)

# РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ НА PYTHON

Отчет по лабораторной работе №3 по дисциплине

«Разработка веб-сервисов для научных и прикладных задач»

Вариант 8

Студент гр. 435-2
_____А.С. Золотухин
«___» _____ 2026 г.

Руководитель: доцент каф. АСУ, канд. техн. наук
_____А.Я. Суханов
«___» _____ 2026 г.

Томск 2026

**Оглавление**

Введение....3
1 СТРУКТУРА ПРОЕКТА....4
2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ....5
2.1 Обработка изображения....5
2.2 Маршруты приложения....5
2.3 Шаблоны страниц и стили....6
2.4 Капча....6
3 ТЕСТИРОВАНИЕ И НЕПРЕРЫВНАЯ ИНТЕГРАЦИЯ....9
3.1 Тесты....9
3.2 GitHub Actions....10
4 ДЕПЛОЙ НА RENDER....11
5 РЕЗУЛЬТАТЫ РАБОТЫ....15
Заключение....18
Список использованных источников....19
Приложение А (обязательное) Листинг программы....20

## Введение

Цель работы – получение навыков разработки веб-приложения на языке Python с использованием фреймворка, системы контроля версий Git, непрерывной интеграции и развёртывания приложения на PaaS-платформе [1].

Задание: реализовать веб-приложение в соответствии с вариантом, используя какой-либо фреймворк. В работе должна использоваться капча. Проект нужно выгрузить на GitHub и настроить простейшее тестирование с помощью CI; при успешном тестировании должен выполняться деплой на PaaS-систему. Входные параметры задаются через html-формы, для приложения задаются стили CSS, результаты размещаются в html-таблицах или с помощью тегов <legend> и <fieldset>.

Вариант 8: веб-приложение должно поменять местами правую и левую части картинки либо верхнюю и нижнюю в зависимости от желания пользователя и нарисовать график распределения цветов исходной картинки.

Приложение написано на фреймворке FastAPI, для защиты формы используется Google reCAPTCHA, тесты запускаются в GitHub Actions, а развёртывание выполнено на платформе Render. Исходный код размещён в репозитории https://github.com/1mrammor/web-services-lab3.

Адрес развёрнутого приложения: https://web-services-lab3-q8in.onrender.com.

# 1 СТРУКТУРА ПРОЕКТА

Структура каталогов и файлов проекта приведена ниже. Код приложения находится в пакете app, html-шаблоны – в папке templates, стили – в папке static, тесты – в папке tests.

```
web-services-lab3/
├── .github/workflows/ci.yml   # тесты в GitHub Actions
├── app/
│   ├── __init__.py
│   ├── main.py                # приложение FastAPI, маршруты
│   ├── image_ops.py           # обработка изображения и график
│   └── captcha.py             # проверка reCAPTCHA на сервере
├── static/styles.css          # стили страниц
├── templates/
│   ├── index.html             # страница с формой
│   └── result.html            # страница с результатом
├── tests/
│   ├── __init__.py
│   └── test_app.py            # тесты
├── .gitignore
├── README.md
└── requirements.txt           # зависимости
```

Зависимости проекта перечислены в файле requirements.txt с точными версиями, чтобы и в CI, и на сервере устанавливались одни и те же библиотеки. Их назначение приведено в таблице 1.1.

Таблица 1.1 – Зависимости проекта

| Библиотека | Назначение |
|---|---|
| `fastapi` | веб-фреймворк: маршруты, обработка форм |
| `uvicorn` | ASGI-сервер, на котором запускается приложение |
| `jinja2` | html-шаблоны страниц |
| `python-multipart` | приём форм с загружаемым файлом |
| `requests` | запрос к Google для проверки капчи |
| `numpy, pillow` | чтение изображения и работа с ним как с массивом |
| `matplotlib` | построение графика распределения цветов |
| `pytest, httpx` | запуск тестов и тестовый клиент |

# 2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

## 2.1 Обработка изображения

Функции обработки находятся в модуле image_ops.py. Загруженный файл открывается библиотекой Pillow, приводится к формату RGB и превращается в массив numpy размерности (высота, ширина, 3).

Перестановку частей выполняет функция swap_halves. Половины меняются местами с помощью срезов массива, без циклов. Для левой и правой частей:

```
half = w // 2
result[:, :half] = img[:, w - half:]
result[:, w - half:] = img[:, :half]
```

Для верхней и нижней частей используются такие же срезы, но по строкам. Если ширина (или высота) нечётная, средний столбец (строка) остаётся на месте, а меняются только крайние половины одинакового размера.

График распределения цветов строит функция color_histogram_png. Для каждого канала r, g, b функция np.bincount подсчитывает, сколько пикселей имеют каждое значение от 0 до 255, и эти три кривые рисуются на одном графике своим цветом. Используется объект Figure из matplotlib без pyplot: так график создаётся независимо для каждого запроса, что безопасно для веб-сервера.

Исходная картинка, результат и график не сохраняются в файлы на сервере: они переводятся в формат PNG, кодируются в base64 и вставляются прямо в страницу в виде data URI. Поэтому приложению не нужна папка для временных файлов.

## 2.2 Маршруты приложения

Приложение FastAPI создаётся в файле main.py. Папка static подключается для отдачи стилей, шаблоны подключаются через Jinja2Templates. В приложении два маршрута:

1) GET / – показывает страницу с формой;

2) POST /process – принимает форму, обрабатывает картинку и показывает результат.

Параметры формы описаны прямо в заголовке функции, FastAPI сам извлекает их из запроса:

```
def process(request: Request,
            direction: Literal["lr", "tb"] = Form(),
            image: UploadFile = File(),
            captcha: str = Form("", alias="g-recaptcha-response")):
```

Тип Literal["lr", "tb"] допускает только два значения направления, при другом значении FastAPI сам вернёт ошибку. Сначала проверяется капча, затем файл открывается как изображение. Если капча не пройдена или вместо картинки загружен другой файл, пользователь снова видит форму с сообщением об ошибке, а сервер возвращает код 400. Если всё в порядке, формируется страница с исходной картинкой, результатом и графиком.

### 2.3 Шаблоны страниц и стили

Страница index.html содержит форму с методом POST и кодировкой multipart/form-data, нужной для передачи файла. В форме два переключателя (radio) для выбора направления – «Левая и правая части» и «Верхняя и нижняя части», поле выбора файла и виджет капчи. Элементы формы сгруппированы тегами <fieldset> и <legend>.

Страница result.html выводит исходную картинку и результат в html-таблице, а график распределения цветов – в отдельном блоке <fieldset>. Стили в файле styles.css ограничивают ширину страницы и центрируют её, задают рамки таблицы, цвет кнопки и сообщения об ошибке, а изображения не выходят за ширину страницы.

### 2.4 Капча

Для защиты формы используется Google reCAPTCHA v2. Виджет подключается на странице скриптом Google. После того как пользователь

отметил «Я не робот», виджет добавляет в форму поле g-recaptcha-response с токеном.

На сервере функция verify_captcha из модуля captcha.py отправляет этот токен вместе с секретным ключом на адрес проверки Google и смотрит поле success в ответе. Пустой токен, ошибка сети или превышение времени ожидания (5 секунд) считаются непройденной капчей.

Ключи капчи не записаны в коде, а берутся из переменных окружения RECAPTCHA_SITE_KEY и RECAPTCHA_SECRET_KEY. Если переменные не заданы, используются официальные тестовые ключи Google, с которыми капча всегда проходит – это удобно для локального запуска.

Для работы на сервере ключи получены в консоли администратора reCAPTCHA (рисунок 2.1). Зарегистрирован сайт lab3-web-services с типом «Версия 2–флажок», в список доменов добавлены localhost для локальной проверки и адрес приложения на Render web-services-lab3-q8in.onrender.com. На других доменах капча с этими ключами работать не будет.

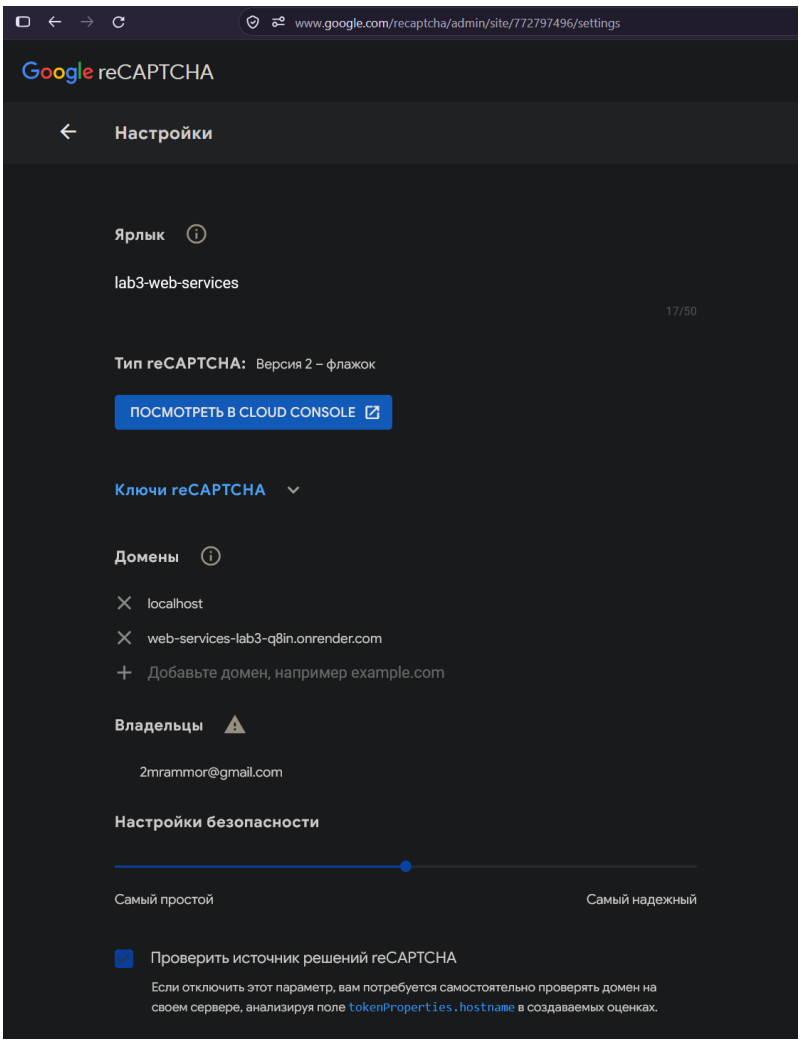



Рисунок 2.1 – Настройки капчи в консоли reCAPTCHA

# 3 ТЕСТИРОВАНИЕ И НЕПРЕРЫВНАЯ ИНТЕГРАЦИЯ

## 3.1 Тесты

Тесты написаны с использованием pytest и находятся в файле test_app.py. Для проверки страниц используется TestClient из FastAPI: он вызывает приложение напрямую, без запуска сервера. Чтобы тесты не обращались к Google, проверка капчи подменяется заглушкой через monkeypatch. Тесты приведены в таблице 3.1.

Таблица 3.1 – Тесты приложения

| Тест | Что проверяет |
|---|---|
| `test_swap_left_right` | столбцы [0, 10, 20, 30] после перестановки дают [20, 30, 0, 10] |
| `test_swap_left_right_odd` | при нечётной ширине средний столбец остаётся на месте |
| `test_swap_top_bottom` | строки 0, 1, 2, 3 после перестановки дают 2, 3, 0, 1 |
| `test_histogram_is_png` | график возвращается в формате PNG |
| `test_index` | главная страница открывается и содержит капчу |
| `test_process` | при пройденной капче в ответе три картинки |
| `test_captcha_failed` | при непройденной капче ответ 400 |
| `test_not_an_image` | при загрузке не картинки ответ 400, сервер не падает |

Результат локального запуска командой python -m pytest -v приведён на рисунке 3.1.

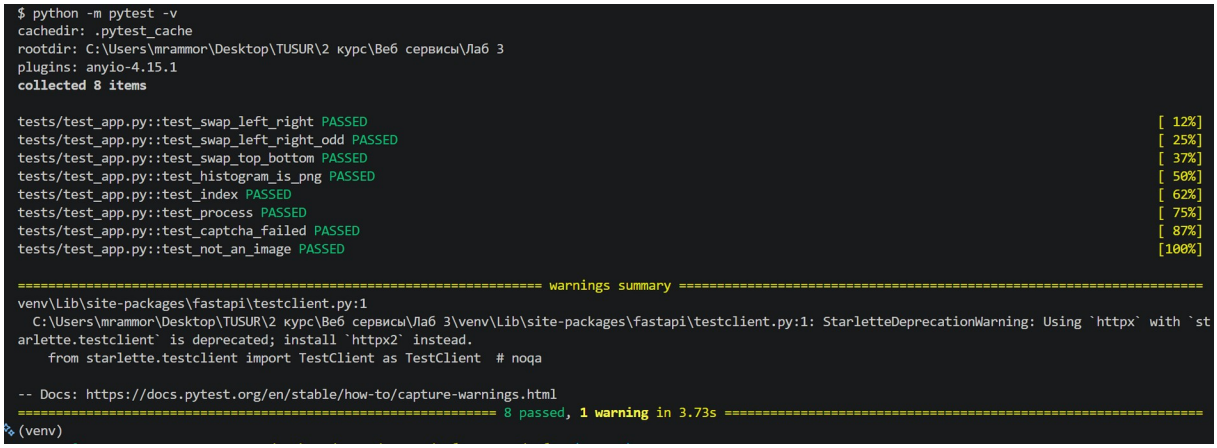



Рисунок 3.1 – Результат запуска тестов

Все восемь тестов прошли. Единственное предупреждение относится к самой библиотеке: в новой версии Starlette тестовый клиент на основе httpx помечен как устаревший. На результат тестов это не влияет.

## 3.2 GitHub Actions

Непрерывная интеграция настроена в файле .github/workflows/ci.yml:

```
name: CI
on: [push]
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: python -m pytest
```

При каждом push в репозиторий GitHub запускает виртуальную машину Ubuntu, скачивает код, устанавливает Python 3.13 и зависимости и выполняет тесты. Если хотя бы один тест не прошёл, проверка считается проваленной. Запуски проверки в GitHub Actions приведены на рисунке 3.2.

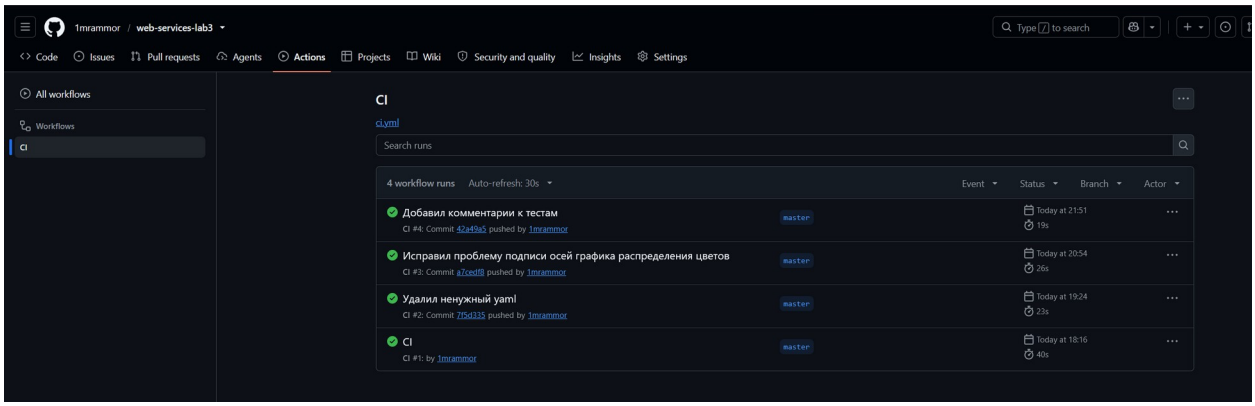


Рисунок 3.2 – Запуски проверки CI в GitHub Actions

Проверка запускалась после каждого push, все четыре запуска завершились успешно (зелёная отметка), каждый занял от 19 до 40 секунд.

# 4 ДЕПЛОЙ НА RENDER

Приложение развёрнуто на платформе Render как Web Service. Настройки заданы в веб-интерфейсе Render. Сервис связан с репозиторием GitHub 1mrammor/web-services-lab3, сборка выполняется из ветки master. Общие настройки сервиса и источник сборки приведены на рисунке 4.1.

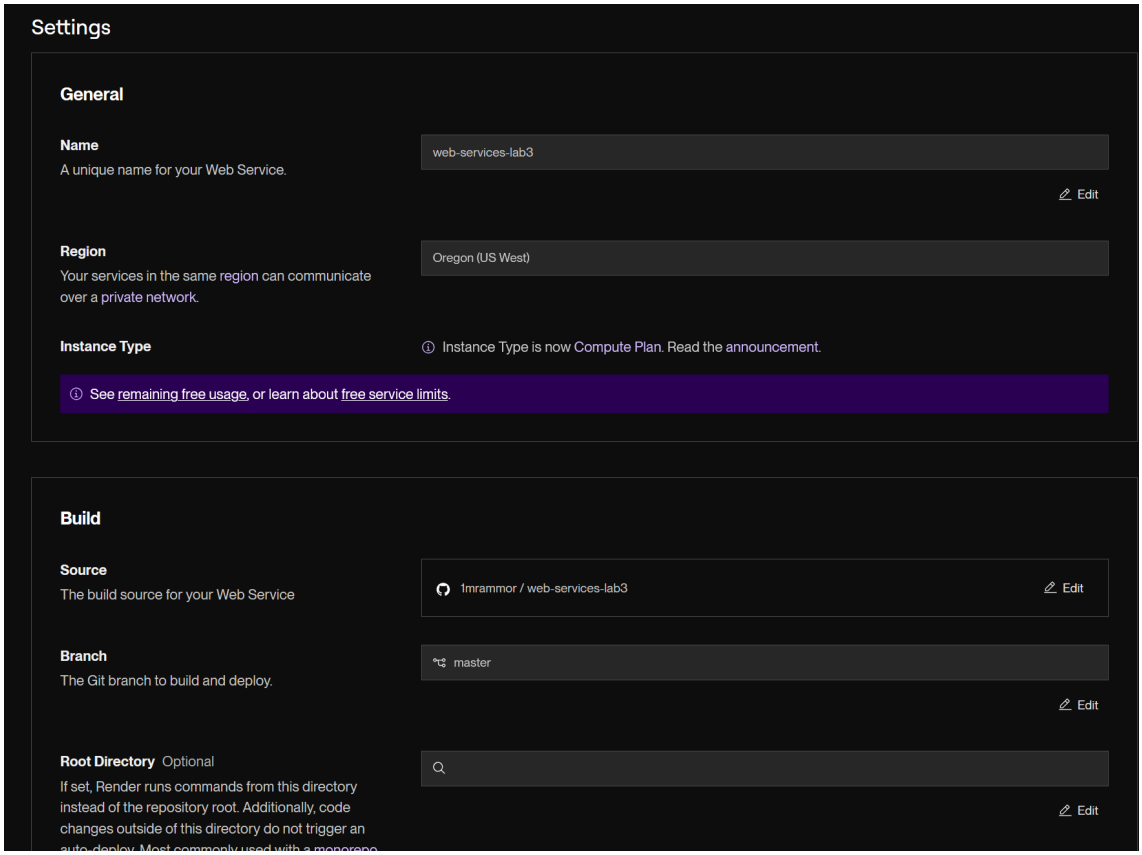



Рисунок 4.1 – Общие настройки сервиса и источник сборки

При каждой сборке Render выполняет команду установки зависимостей pip install -r requirements.txt (рисунок 4.2). Поле Root Directory оставлено пустым, так как приложение находится в корне репозитория.

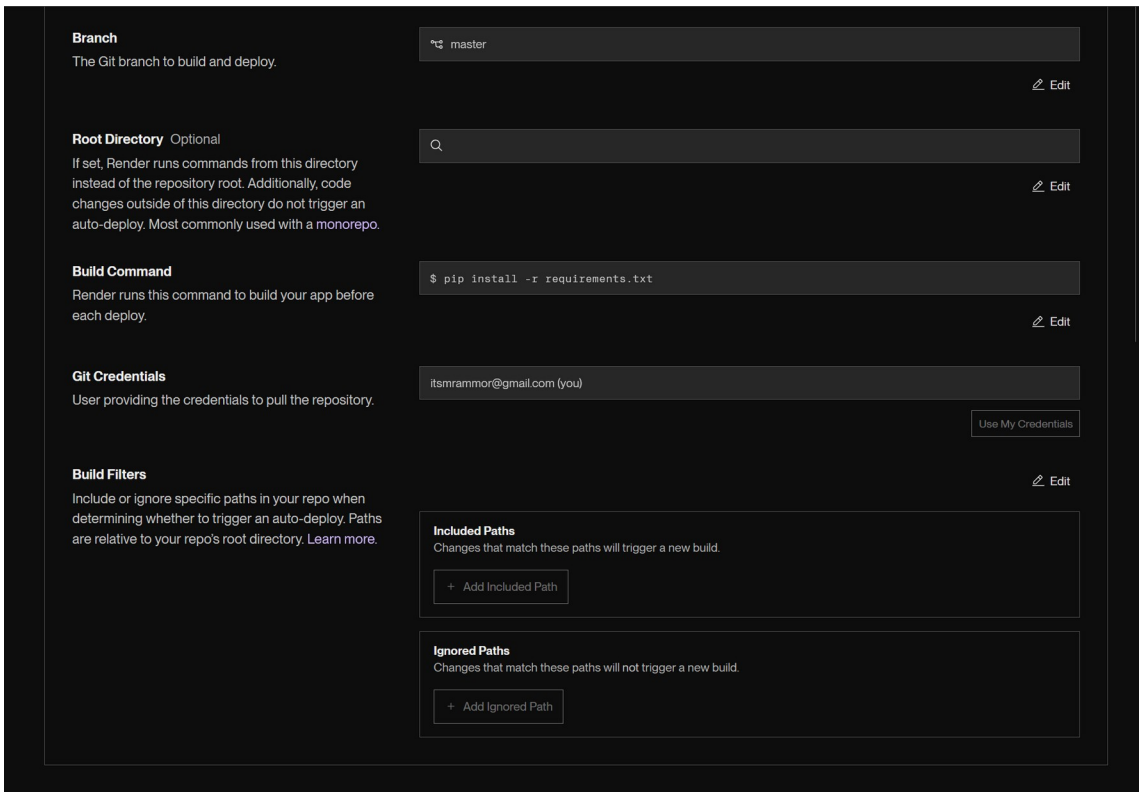



Рисунок 4.2 – Команда сборки

Приложение запускается командой:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Здесь app.main:app – объект app в модуле app/main.py. Адрес 0.0.0.0 нужен, чтобы сервер принимал запросы извне контейнера, а номер порта Render передаёт в переменной окружения PORT, поэтому порт не записывается вручную.

Для автоматического деплоя выбран режим After CI Checks Pass (рисунок 4.3): после push Render ждёт завершения проверок GitHub Actions и разворачивает новую версию, только если тесты прошли успешно. Так выполняется требование задания о деплое при успешном тестировании. Deploy Hook – секретная ссылка для ручного запуска деплоя, поэтому её значение скрыто.

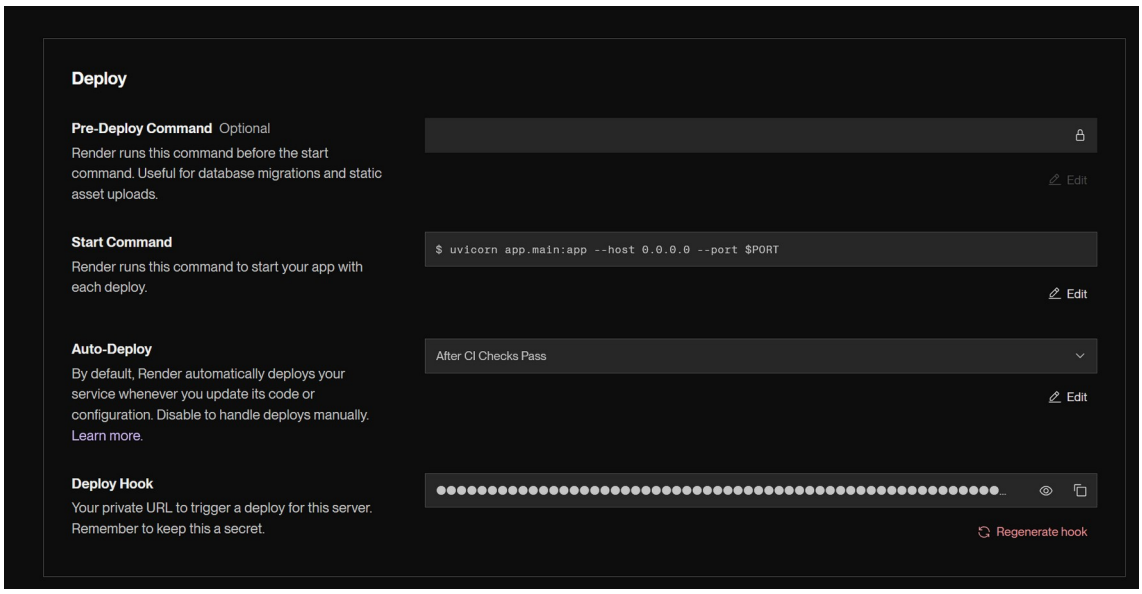



Рисунок 4.3 – Команда запуска и автоматический деплой

Ключи капчи заданы на Render в переменных окружения сервиса (вкладка Environment):

```
RECAPTCHA_SITE_KEY=<публичный ключ сайта>
RECAPTCHA_SECRET_KEY=<секретный ключ>
```

Значения ключей в отчёте не приводятся, так как секретный ключ нельзя публиковать.

События деплоя приведены на рисунке 4.4. После push коммита 42a49a5 Render автоматически начал деплой (New commit via Auto-Deploy), а через две минуты новая версия стала доступна (Deploy live). Сервис работает в статусе Live по адресу https://web-services-lab3-q8in.onrender.com.

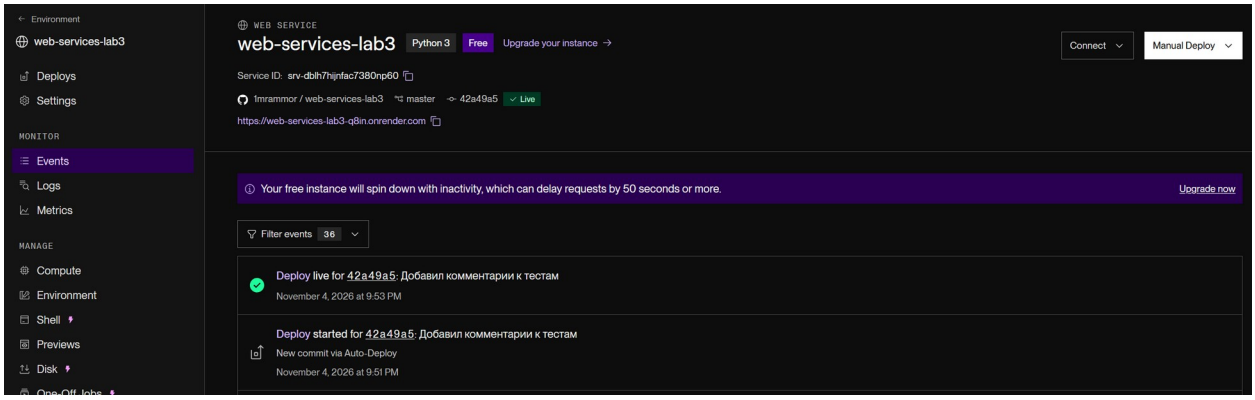



Рисунок 4.4 – События деплоя на Render

В журнале приложения (рисунок 4.5) Render сообщает, что сервис запущен на порту 10000 – именно это значение подставляется в переменную

PORT. Записи Shutting down означают, что бесплатный сервис остановился после периода бездействия. При следующем запросе он запускается снова, поэтому первый ответ может приходить с задержкой до 50 секунд, о чём Render предупреждает в верхней части страницы.

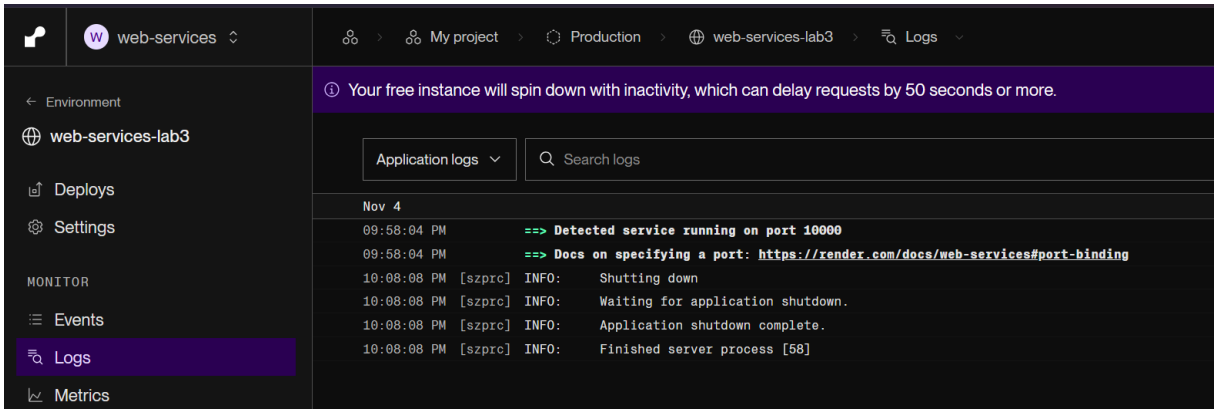



Рисунок 4.5 – Журнал приложения на Render

# 5 РЕЗУЛЬТАТЫ РАБОТЫ

Главная страница развёрнутого приложения с формой приведена на рисунке 5.1. Пользователь выбирает направление перестановки, загружает картинку и проходит капчу.

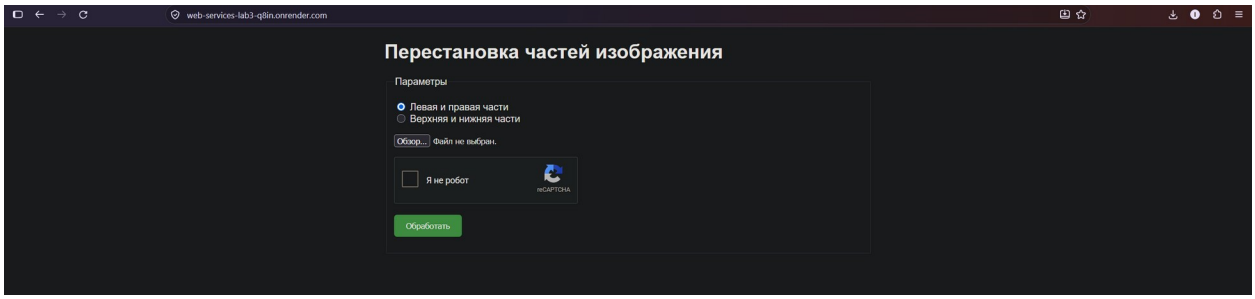



Рисунок 5.1 – Главная страница приложения

Результат перестановки левой и правой частей приведён на рисунке 5.2, верхней и нижней частей – на рисунке 5.4. На странице результата в таблице показаны исходная картинка и картинка после перестановки, а ниже – график распределения цветов исходной картинки.

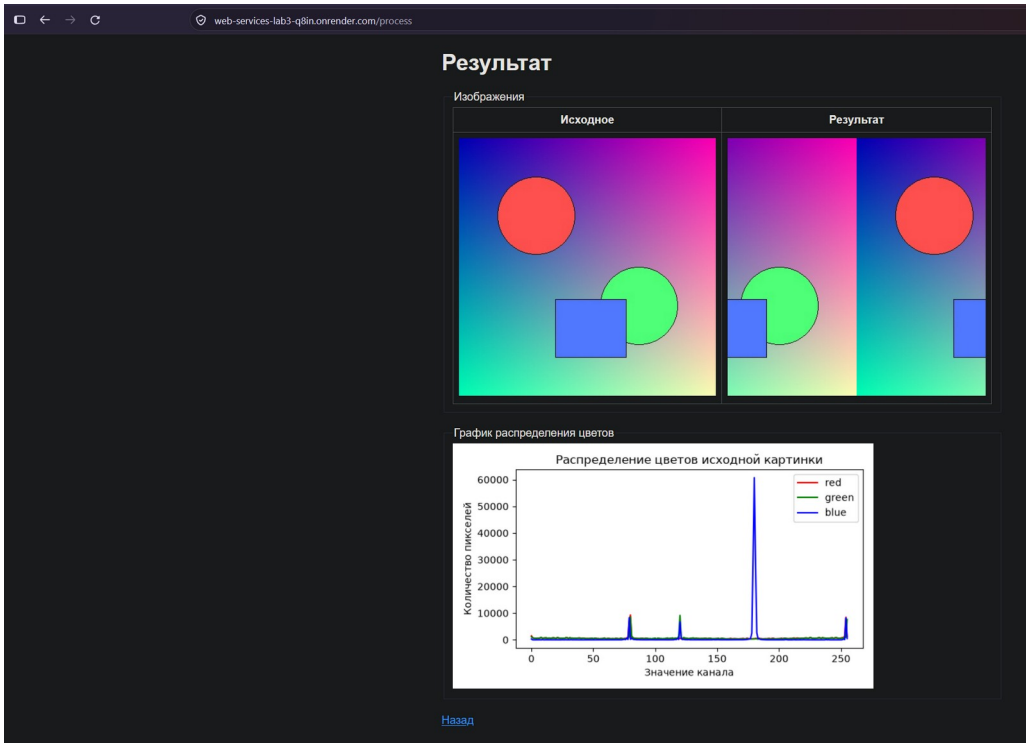



Рисунок 5.2 – Перестановка левой и правой частей картинки

Красный круг из левой половины картинки переместился вправо, а зелёный круг и квадрат из правой половины – влево. Квадрат стоял на границе половин, поэтому он разрезан на две части по краям картинки.

Для перестановки верхней и нижней частей в форме выбирается второй вариант и проходится капча (рисунок 5.3).

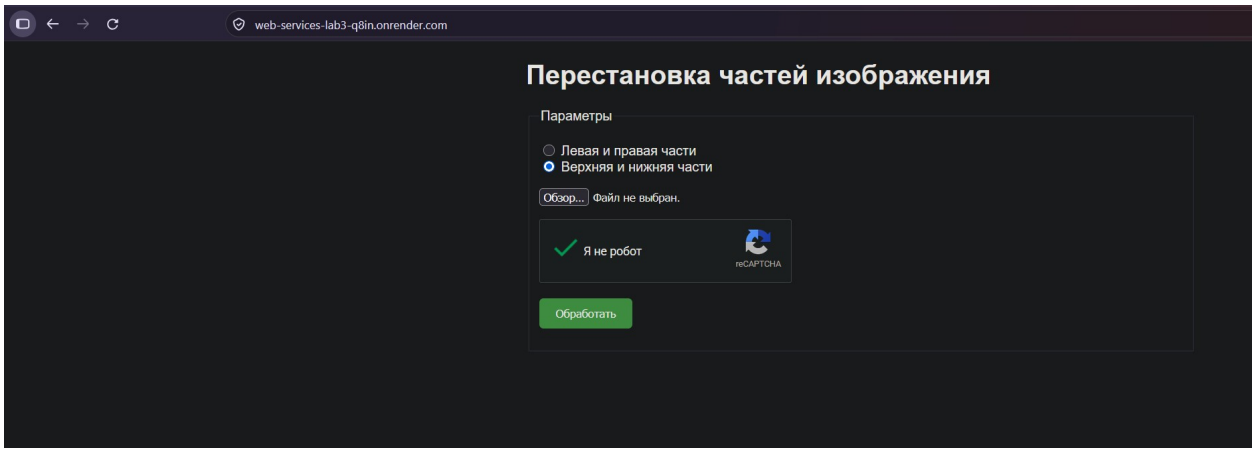



Рисунок 5.3 – Форма с выбранной перестановкой верхней и нижней частей

Результат перестановки верхней и нижней частей приведён на рисунке 5.4.

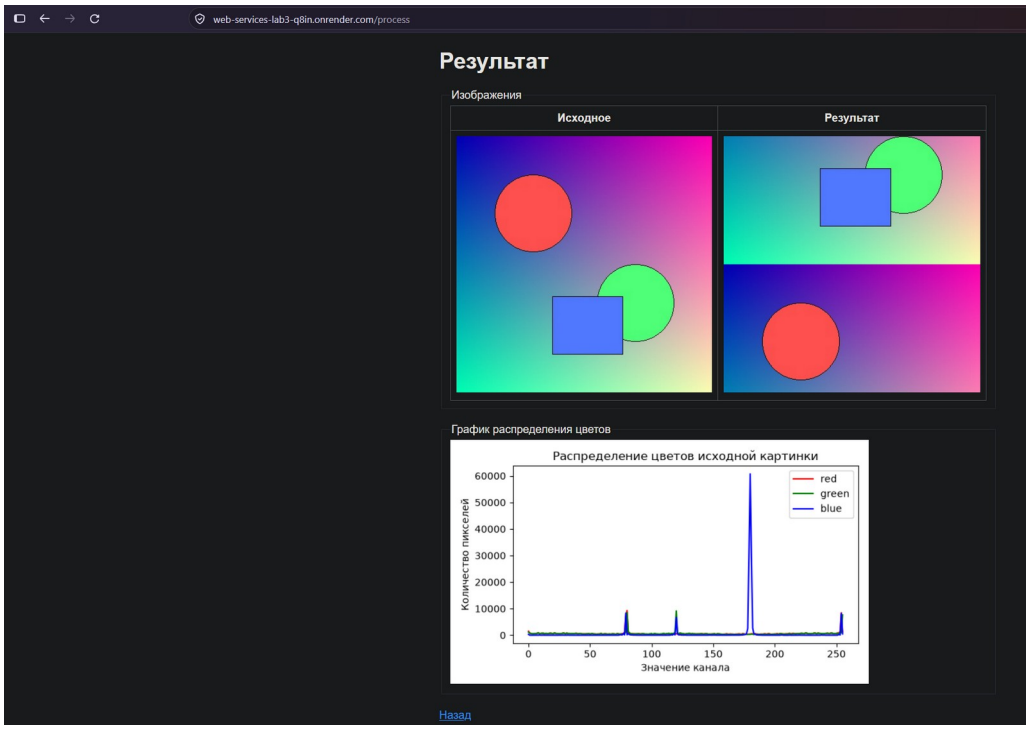



Рисунок 5.4 – Перестановка верхней и нижней частей картинки

Здесь нижняя половина с зелёным кругом и квадратом оказалась сверху, а верхняя половина с красным кругом – снизу.

На графике три кривые – красный, зелёный и синий каналы: по горизонтали отложено значение канала от 0 до 255, по вертикали – количество пикселей с таким значением. График строится по исходной картинке, а перестановка частей распределение цветов не меняет, так как пиксели только перемещаются.

Если отправить форму, не пройдя капчу, приложение снова показывает форму с сообщением «Капча не пройдена»; этот случай проверяется тестом test_captcha_failed.

## Заключение

В ходе лабораторной работы разработано веб-приложение на FastAPI, которое меняет местами левую и правую или верхнюю и нижнюю части картинки по выбору пользователя и строит график распределения цветов исходной картинки.

Входные параметры задаются через html-форму, результаты выводятся в таблице и блоках <fieldset>, для страниц заданы стили CSS. Форма защищена капчей Google reCAPTCHA с проверкой на сервере, ключи хранятся в переменных окружения.

Проект размещён на GitHub. Для него написаны восемь тестов, которые автоматически запускаются в GitHub Actions при каждом push. Приложение развёрнуто на Render, причём новая версия разворачивается только после успешного прохождения тестов.

## Список использованных источников

1. Суханов А. Я. Разработка веб-сервисов для научных и прикладных задач: методические указания по выполнению лабораторных работ и самостоятельной работе. Томск: ТУСУР, 2023. 113 с.

2. FastAPI documentation [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 04.11.2026).

3. Render Docs. Deploy a FastAPI App [Электронный ресурс]. URL: https://render.com/docs/deploy-fastapi (дата обращения: 04.11.2026).

4. GitHub Docs. GitHub Actions [Электронный ресурс]. URL: https://docs.github.com/actions (дата обращения: 04.11.2026).

5. Google reCAPTCHA. Developer’s Guide [Электронный ресурс]. URL: https://developers.google.com/recaptcha (дата обращения: 04.11.2026).

# Приложение А

(обязательное)

## Листинг программы

### А.1 Файл app/main.py

```
from pathlib import Path
from typing import Literal

from fastapi import FastAPI, File, Form, Request, UploadFile
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from app.captcha import site_key, verify_captcha
from app.image_ops import (array_to_data_uri, color_histogram_png,
load_image,
                           png_to_data_uri, swap_halves)

BASE_DIR = Path(__file__).resolve().parent.parent

app = FastAPI()
app.mount("/static", StaticFiles(directory=BASE_DIR / "static"),
name="static")
templates = Jinja2Templates(directory=BASE_DIR / "templates")


def show_form(request, error=None, status_code=200):
    context = {"site_key": site_key(), "error": error}
    return templates.TemplateResponse(request, "index.html", context,
status_code=status_code)


@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return show_form(request)


@app.post("/process", response_class=HTMLResponse)
def process(
    request: Request,
    direction: Literal["lr", "tb"] = Form(),
    image: UploadFile = File(),
    captcha: str = Form("", alias="g-recaptcha-response"),
):
    if not verify_captcha(captcha):
        return show_form(request, "Капча не пройдена", 400)

    try:
        original = load_image(image.file.read())
    except OSError:
        return show_form(request, "Файл не является изображением", 400)

    context = {
        "original": array_to_data_uri(original),
        "result": array_to_data_uri(swap_halves(original, direction)),
        "histogram": png_to_data_uri(color_histogram_png(original)),
```

```
    }
    return templates.TemplateResponse(request, "result.html", context)
```

## A.2 Файл app/image_ops.py

```
import base64
import io

import numpy as np
from matplotlib.figure import Figure
from PIL import Image


def load_image(data):
    return np.array(Image.open(io.BytesIO(data)).convert("RGB"))


def swap_halves(img, direction):
    result = img.copy()
    if direction == "lr":
        w = img.shape[1]
        half = w // 2
        result[:, :half] = img[:, w - half:]
        result[:, w - half:] = img[:, :half]
    else:
        h = img.shape[0]
        half = h // 2
        result[:half] = img[h - half:]
        result[h - half:] = img[:half]
    return result


def color_histogram_png(img):
    fig = Figure(figsize=(6, 3.5))
    ax = fig.subplots()
    for i, color in enumerate(("red", "green", "blue")):
        counts = np.bincount(img[:, :, i].ravel(), minlength=256)
        ax.plot(counts, color=color, label=color)
    ax.set_xlabel("Значение канала")
    ax.set_ylabel("Количество пикселей")
    ax.set_title("Распределение цветов исходной картинки")
    ax.legend()
    fig.tight_layout()
    buf = io.BytesIO()
    fig.savefig(buf, format="png")
    return buf.getvalue()


def png_to_data_uri(png_bytes):
    return "" + base64.b64encode(png_bytes).decode()


def array_to_data_uri(arr):
    buf = io.BytesIO()
    Image.fromarray(arr).save(buf, format="PNG")
    return png_to_data_uri(buf.getvalue())
```

## A.3 Файл app/captcha.py

```
import os

import requests

VERIFY_URL = "https://www.google.com/recaptcha/api/siteverify"
TEST_SITE_KEY = "6LeIxAcTAAAAAJcZVRqyHh71UMIEGNQ_MXjiZKhI"
TEST_SECRET_KEY = "6LeIxAcTAAAAAGG-vFI1TnRWxMZNFuojJ4WifJWe"


def site_key():
    return os.environ.get("RECAPTCHA_SITE_KEY", TEST_SITE_KEY)


def verify_captcha(token):
    if not token:
        return False
    secret = os.environ.get("RECAPTCHA_SECRET_KEY", TEST_SECRET_KEY)
    try:
        response = requests.post(VERIFY_URL, data={"secret": secret,
"response": token}, timeout=5)
        return response.json().get("success", False)
    except requests.RequestException:
        return False
```

## A.4 Файл templates/index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Перестановка частей изображения</title>
    <link rel="stylesheet" href="/static/styles.css">
    <script src="https://www.google.com/recaptcha/api.js" async
defer></script>
</head>
<body>
    <h1>Перестановка частей изображения</h1>

    {% if error %}
    <p class="error">{{ error }}</p>
    {% endif %}

    <form method="post" action="/process" enctype="multipart/form-data">
        <fieldset>
            <legend>Параметры</legend>
            <p>
                <label><input type="radio" name="direction" value="lr"
checked> Левая и правая части</label><br>
                <label><input type="radio" name="direction" value="tb">
Верхняя и нижняя части</label>
            </p>
            <p><input type="file" name="image" accept="image/*" required></p>
            <div class="g-recaptcha" data-sitekey="{{ site_key }}"></div>
            <p><button type="submit">Обработать</button></p>
        </fieldset>
    </form>
```

```
</body>
</html>
```

## A.5 Файл templates/result.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Результат</title>
    <link rel="stylesheet" href="/static/styles.css">
</head>
<body>
    <h1>Результат</h1>

    <fieldset>
        <legend>Изображения</legend>
        <table>
            <tr>
                <th>Исходное</th>
                <th>Результат</th>
            </tr>
            <tr>
                <td><img src="{{ original }}"></td>
                <td><img src="{{ result }}"></td>
            </tr>
        </table>
    </fieldset>

    <fieldset>
        <legend>График распределения цветов</legend>
        <img src="{{ histogram }}">
    </fieldset>

    <p><a href="/">Назад</a></p>
</body>
</html>
```

## A.6 Файл static/styles.css

```
body {
    max-width: 800px;
    margin: 20px auto;
    padding: 0 16px;
    font-family: Arial, sans-serif;
}

fieldset {
    margin-bottom: 20px;
}

table {
    border-collapse: collapse;
    width: 100%;
}

th, td {
    border: 1px solid #999;
    padding: 8px;
```

```
    text-align: center;
}

img {
    max-width: 100%;
}

button {
    background-color: #4CAF50;
    color: white;
    border: none;
    border-radius: 5px;
    padding: 10px 20px;
    cursor: pointer;
}

.error {
    color: #b00020;
}
```

## A.7 Файл tests/test_app.py

```
import io

import numpy as np
from fastapi.testclient import TestClient
from PIL import Image

from app import main
from app.image_ops import color_histogram_png, swap_halves

# "виртуальный браузер": вызывает приложение напрямую, сервер запускать не
нужно
client = TestClient(main.app)


# тестовая картинка: height строк, width столбцов, столбец x залит числом x *
10
# для width=4 значения по столбцам: [0, 10, 20, 30]
def make_image(width, height=2):
    img = np.zeros((height, width, 3), dtype=np.uint8)
    for x in range(width):
        img[:, x] = x * 10
    return img


# превращает массив в байты PNG, как будто это загруженный файл
def png_bytes(arr):
    buf = io.BytesIO()
    Image.fromarray(arr).save(buf, format="PNG")
    return buf.getvalue()


# отправляет форму на /process эндпоинт, как это делает браузер
# direction - "lr" или "tb", data - байты файла,
# captcha_ok - подменяет проверку капчи на заглушку (True - прошла, False -
не прошла)
def post(direction, data, monkeypatch, captcha_ok=True):
    monkeypatch.setattr(main, "verify_captcha", lambda token: captcha_ok)
    return client.post(
        "/process",
```

```
        data={"direction": direction, "g-recaptcha-response": "token"},
        files={"image": ("test.png", data, "image/png")},
    )


# левая и правая половины меняются местами: [0, 10, 20, 30] -> [20, 30, 0,
10]
def test_swap_left_right():
    result = swap_halves(make_image(4), "lr")
    assert result[0, :, 0].tolist() == [20, 30, 0, 10]


# нечётная ширина: средний столбец (20) остаётся на месте
def test_swap_left_right_odd():
    result = swap_halves(make_image(5), "lr")
    assert result[0, :, 0].tolist() == [30, 40, 20, 0, 10]


# верхняя и нижняя половины: строки 0, 1, 2, 3 -> 2, 3, 0, 1
def test_swap_top_bottom():
    img = np.zeros((4, 3, 3), dtype=np.uint8)
    for y in range(4):
        img[y] = y
    result = swap_halves(img, "tb")
    assert result[:, 0, 0].tolist() == [2, 3, 0, 1]


# график возвращается как PNG (любой PNG начинается с байтов \x89PNG)
def test_histogram_is_png():
    assert color_histogram_png(make_image(4)).startswith(b"\x89PNG")


# GET / : главная страница открывается (200) и на ней есть капча
def test_index():
    response = client.get("/")
    assert response.status_code == 200
    assert "g-recaptcha" in response.text


# POST /process с картинкой и пройденной капчей: 200 и в ответе 3 картинки
# (исходная, результат, график)
def test_process(monkeypatch):
    response = post("lr", png_bytes(make_image(4)), monkeypatch)
    assert response.status_code == 200
    assert response.text.count("data:image/png;base64") == 3


# капча не пройдена (captcha_ok=False): ответ 400
def test_captcha_failed(monkeypatch):
    response = post("lr", png_bytes(make_image(4)), monkeypatch,
captcha_ok=False)
    assert response.status_code == 400


# вместо картинки прислали текст: ответ 400, сервер не падает
def test_not_an_image(monkeypatch):
    response = post("lr", b"not an image", monkeypatch)
    assert response.status_code == 400
```

## A.8 Файл .github/workflows/ci.yml

```
name: CI

on: [push]

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: python -m pytest
```

## A.9 Файл requirements.txt

```
fastapi==0.162.0
uvicorn==0.54.0
jinja2==3.1.6
python-multipart==0.0.32
requests==2.34.2
numpy==2.5.4
pillow==13.0.0
matplotlib==3.11.2
pytest==9.2.0
httpx==0.28.1
```